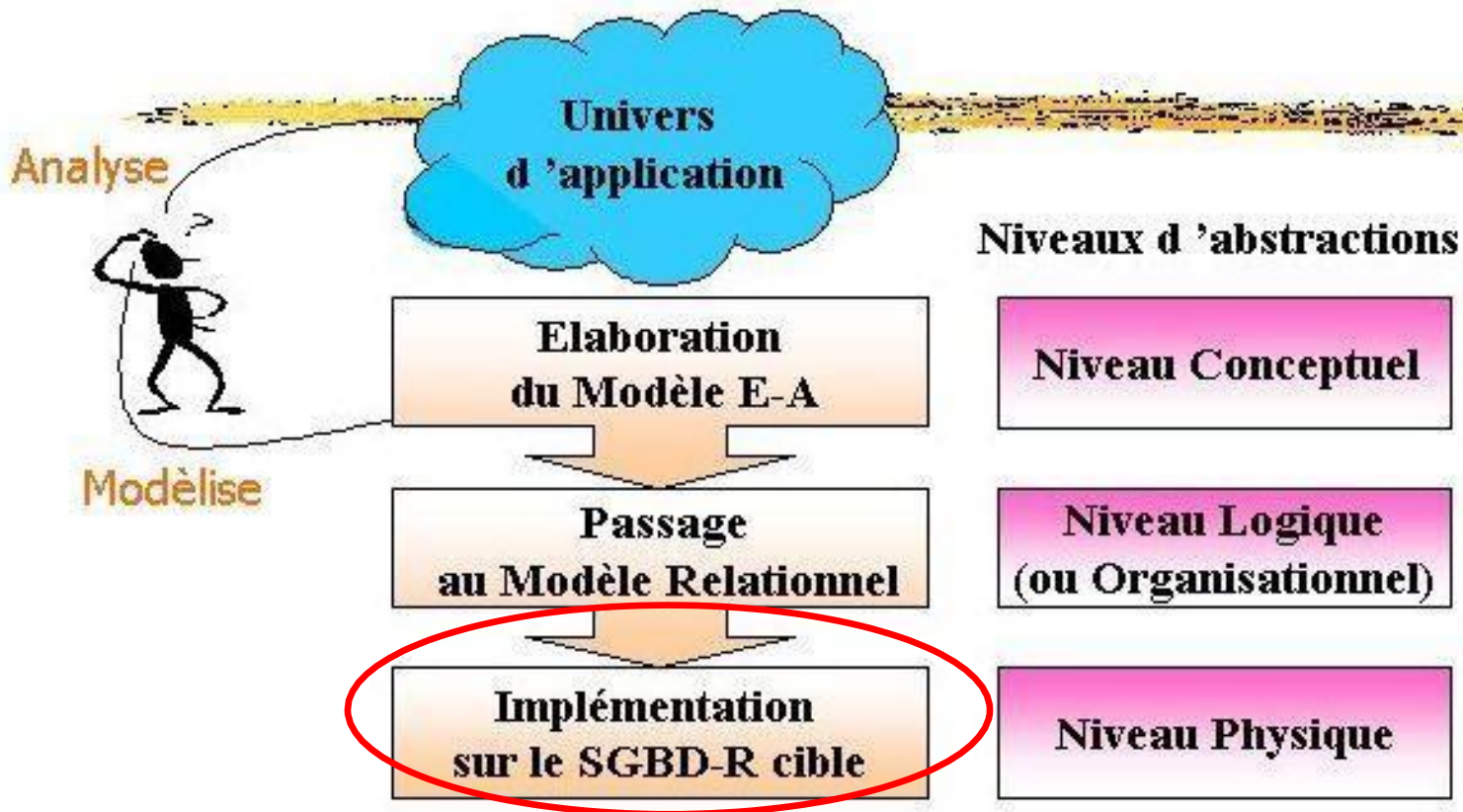


COURS SYSTÈMES DE GESTION DES BASES DE DONNÉES - PARTIE I SQL AVANCE

Réalisé par : Mme YEKKEN Sabine

Version : 2012 -2013

Introduction générale.



- Pour interroger une BD, il faut utiliser un langage .
- **SQL** est le langage le plus complet pour interagir une BD.

CHAPITRE 1

Introduction SQL Langage de définition de données

Objectifs du cours

(1/2)

Introduction du langage SQL

1^{ème} Partie : Présentation du langage LDD

1. Définir « LDD »
2. Les types de données

2^{ème} Partie : Création d'une table/des contraintes d'intégrités

1. Créer une table/ table à partir d'une autre
2. Ajouter une contrainte clé primaire, clé étrangère
3. Ajouter une contrainte (On delete cascade, on delete set null)
4. Autres contraintes (not null, check, unique)
5. Activer / désactiver une contrainte

Objectifs du cours

(2/2)

3^{ème} Partie : Manipulation de la structure de la table

1. Ajouter, modifier, supprimer une colonne/contrainte
2. Supprimer une table
3. Renommer une table

4^{ème} Partie : Autres Objets

1. Définition et création de :
 - i. vues
 - ii. Synonymes
 - iii. Index
 - iv. Séquences

Introduction du langage SQL

Objectifs de la partie :

Introduire et définir le langage SQL.

Définition de l'SQL.

SQL (**S**tructured **Q**uery **L**anguage), en français langage de requête structurée, est composé d'un ensemble de sous langages:

- un langage de définition de données (**LDD**),
- un langage de manipulation de données (**LMD**),
- un langage d'interrogation de données (**LID**),
- un langage de contrôle de données (**LCD**),

pour les bases de données relationnelles.

1^{ère} Partie :

Présentation du langage LDD

Objectifs de la partie :

Définition et présentation du langage de définition de données ainsi que les différents types de données.

LDD.

Le langage de définition de données, c'est un langage qui permet de **définir** et **manipuler** les structures de données et non pas les données.

Structure de données : tout objet de la BD destiné à contenir des données : Exemple : TABLE.

Les types de données.

Types de données	Description
INT	Entier
CHAR [(size [BYTE CHAR])]	Taille fixe comprise entre 1 et 2000
VARCHAR2 (size)	Taille Variable comprise entre 1 et 4000
NUMBER [(t , d)]	Réel de t chiffres dont d sont décimales.
LONG	Données caractères ayant une taille <= 2GO
DATE	Date comprise entre 1/1/4712 AJC et 31/12/999 APJC
TIMESTAMP	Année, mois, jour , heure, minute et seconde, fraction de seconde
INTERVAL YEAR(year_precision) TO MONTH	Stocke une période de temps en année et mois, où year_precision est compris entre 0 et 9. La valeur par défaut est 2

2ème et 3ème Partie:

Création d'une table/contraintes d'intégrités -
Manipulation de la structure d'une table

Objectifs de la partie :

Comment créer une table et définir les contraintes d'intégrités.

Comment ajouter, modifier ou supprimer des colonnes ou des contraintes...

Créer une table. **Syntaxe**

Syntaxe :

```
CREATE TABLE <nom_de_la_table>  
  
    ( <nom_colonne1> <type_colonne1>,  
      <nom_colonne2> <type_colonne2>,    ...  
      <nom_colonne'> <type_colonne'>  
  
    );
```

Remarque : Pour créer une table il faut avoir :

- Le privilège **CREATE TABLE**
- Un espace de stockage

- Type d'objet TABLE Objet ETUDIANTS

[illegible]

Créer une table. Table à partir d'une autre table

- **CREATE TABLE** Etud **AS** SELECT * From Etudiants ;

la table Etud aura la même structure de la table Etudiants avec tous les enregistrements de la table.

→ la table Etud aura la même structure de la table Etudiants **avec tous** les enregistrements de la table.

- **CREATE TABLE** Etud **AS** SELECT * From Etudiants WHERE (0=1);

→ la table Etud aura la même structure de la table Etudiants **sans** les enregistrements de la table.

- **DESCRIBE** (ou **DESC**) Etud ; --pour voir la description de la table

Ajouter une contrainte clé primaire - Syntaxe (1/3)

1^{er} cas : clé primaire simple

Syntaxe 1: lors de la création de la table:

1^{ère} Méthode : Contrainte niveau COLONNE

CREATE TABLE <nom_de_la_table>

(<nom_COL1 > <type_COL1> **CONSTRAINT** <nom_contrainte> **PRIMARY KEY**,

<nom_COL2 > <type_COL2>, ...

<nom_coln > <type_coln>

);

Ajouter une contrainte clé primaire - Syntaxe (2/3)

Syntaxe 1: lors de la création de la table:

2^{ème} Méthode : Contrainte niveau TABLE

CREATE TABLE <nom_de_la_table>

(<nom_COL1 > <type_COL1> ,

<nom_COL2 > <type_COL2> , ...

<nom_coln > <type_coln> ,

CONSTRAINT <nom_contrainte> PRIMARY KEY(colonne(s)),
);

Ajouter une contrainte clé primaire - Syntaxe (3/3)

2ème cas : clé primaire composée

Syntaxe 1: lors de la création de la table:

```
CREATE TABLE <nom_de_la_table>
```

```
( <nom_COL1 > <type_COL1>,
```

```
  <nom_COL2> <type_COL2>,
```

```
  ...
```

```
  <nom_coln> <type_coln>,
```

```
  Constraint <nom_contrainte> Primary key (NOMCOL1, nomCOL2...) );
```

→ Contrainte niveau Table

Ajouter une contrainte clé primaire - Exemple

Syntaxe :

ALTER TABLE <nom_de_la_table> **ADD Constraint** <nom_contrainte>
PRIMARY KEY (nom_colonne(s))

ALTER TABLE Etudiants **ADD Constraint** pk_nom_prenom **PRIMARY KEY**
 (nom,prenom) ;

Type d'objet TABLE Objet ETUDIANTS

Table	Column	Type De Données	Longueur	Précision	Echelle	Clé Primaire	Valeur Nullable	Valeur Par Défaut	Commentaire
ETUDIANTS	NETUDIANT	Number	-	-	-	-	✓	-	-
	NOM	Varchar2	10	-	-	1	-	-	-
	PRENOM	Varchar2	10	-	-	2	-	-	-

Supprimer une contrainte clé primaire.

Syntaxe :

ALTER TABLE <nom_de_la_table> **DROP CONSTRAINT**
 <nom_contrainte> ;

ALTER TABLE Etudiants **DROP CONSTRAINT** pk_nom_prenom ;

Type d'objet TABLE Objet ETUDIANTS

Table	Column	Type De Données	Longueur	Précision	Echelle	Clé Primaire	Valeur Nullable	Valeur Par Défaut	Commentaire
ETUDIANTS	NETUDIAN	Number	-	-	-	-	✓	-	-
	NOM	Varchar2	10	-	-	-	✓	-	-
	PRENOM	Varchar2	10	-	-	-	✓	-	-

Ajouter une contrainte clé étrangère.

Syntaxe :

```
ALTER TABLE <nom_de_la_table> ADD constraint  
<nom_contrainte> FOREIGN KEY (<nom_cols>) references  
<table_référencée> (nom_cols) ;
```

Create table Etud as select * from Etudiants;

Alter table Etud ADD constraint pk_nom_prenom PRIMARY KEY (nom,
prenom) ;

```
ALTER TABLE Etud ADD constraint fk_etud_etudiants FOREIGN KEY  
(nom,prenom) references Etudiants (nom,prenom) ;
```

Ajouter une contrainte clé étrangère. Options

Constraint <nom_contrainte> **Foreign Key** (<colonne(s)>)

References <TableRéféréncée> (<colonne(s)>)

ON DELETE CASCADE | SET NULL

- La clause **ON DELETE CASCADE** permet de supprimer automatiquement les valeurs d'une clé étrangère dépendantes d'une clé primaire si une valeur de cette dernière vient d'être supprimée.
- La clause **ON DELETE SET NULL** permet de mettre à Zéro automatiquement les valeurs d'une clé étrangère dépendantes d'une clé primaire si une valeur de cette dernière vient d'être supprimée.

Ajouter une contrainte <NOT NULL>.

Cette contrainte est définie :

- Soit lors de la création de la table, syntaxe :

```
<nom_colonne> <type_colonne> constraint <nom_contrainte> NOT NULL
```

- Soit après la création de la table, syntaxe :

```
Alter table <nom_tab> modify <nom_col> constraint < nom_contrainte > NOT NULL
```

La contrainte NOT NULL ne peut être définie qu'au niveau de la colonne, pas au niveau de la table

Ajouter une contrainte <UNIQUE> - Définition

Une **contrainte d'intégrité** de type clé **unique** exige que chaque valeur dans une colonne ou dans un ensemble de colonnes constituant une clé soit unique.

Remarque : Une contrainte unique autorise la valeur NULL à moins que vous définissiez des contraintes NOT NULL

Ajouter une contrainte <UNIQUE> - Syntaxe

Cette contrainte est définie :

- Soit lors de la création de la table, syntaxe :

```
<nom_colonne> <type_colonne> constraint <nom_de_contrainte> UNIQUE
```

- Soit après la création de la table, syntaxe :

```
Alter table <nom_table> add constraint <nom_de_contrainte> UNIQUE  
(<nom_colonne>)
```


Ajouter une contrainte <CHECK>.

La contrainte Check définit une condition que chaque ligne doit vérifier

Cette contrainte est définie :

- Soit lors de la création de la table, syntaxe :

```
<nom_colonne> <type_col> CONSTRAINT <nom_de_contr> CHECK (condition)
```

- Soit après la création de la table, syntaxe :

```
Alter table <nom_table> ADD CONSTRAINT <nom_de_contr> CHECK (condition)
```

Gérer des colonnes et/ou des contraintes - Syntaxes

Ajouter une/plusieurs colonne(s). Exemple

Ajouter 2 colonnes à la table « Etudiants »

```
ALTER TABLE Etudiants ADD (telephone number ,email varchar2(50) );
```

Type d'objet TABLE Objet ETUDIANTS						
Table	Column	Type De Données	Longueur	Précision	Echelle	Clé Primaire
<u>ETUDIANTS</u>	<u>NETUDIANT</u>	Number	-	-	-	-
	<u>NOM</u>	Varchar2	10	-	-	1
	<u>PRENOM</u>	Varchar2	10	-	-	2
	<u>TELEPHONE</u>	Number	-	-	-	-
	<u>EMAIL</u>	Varchar2	50	-	-	-

Modifier une colonne. Exemple

Syntaxe :

ALTER TABLE <nom_de_la_table> **MODIFY** <nouvelle définition de la colonne>

ALTER TABLE Etudiants **MODIFY** (email varchar2(100));

Type d'objet TABLE Objet ETUDIANTS

Table	Column	Type De Données	Longueur	Précision	Echelle	Clé Primaire
<u>ETUDIANTS</u>	<u>NETUDIAN</u>	Number	-	-	-	-
	<u>NOM</u>	Varchar2	10	-	-	1
	<u>PRENOM</u>	Varchar2	10	-	-	2
	<u>TELEPHONE</u>	Number	-	-	-	-
	<u>EMAIL</u>	Varchar2	100	-	-	-

Supprimer une colonne. Exemple

Syntax :

ALTER TABLE <nom_de_la_table> **DROP** <colonnes>

ALTER TABLE Etudiants **DROP** (email, telephone);

Type d'objet	TABLE	Objet	ETUDIANTS						
Table	Column	Type De Données	Longueur	Précision	Echelle	Clé Primaire	Valeur Nullable	Valeur Par Défaut	Commentaire
<u>ETUDIANTS</u>	<u>NETUDIANT</u>	Number	-	-	-	-	✓	-	-
	<u>NOM</u>	Varchar2	10	-	-	-	✓	-	-
	<u>PRENOM</u>	Varchar2	10	-	-	-	✓	-	-
									1 - 3

Activer/Désactiver une contrainte.

Syntaxe :

```
ALTER TABLE <nom_de_la_table>
```

```
ENABLE | DISABLE CONSTRAINT <nom_de_la_contrainte>;
```

Supprimer une table. DROP / TRUNCATE

Syntaxe :

```
DROP TABLE <nom_de_la_table> ;
```

Remarque:

On a aussi la commande « TRUNCATE » qui permet de vider la table

```
TRUNCATE TABLE <nom_de_la_table> ;
```

Renommer une table.

Syntaxe :

```
RENAME <ancien_nom> TO <nouveau_nom> ;
```

Exemple:

```
RENAME Etudiants TO Etudiants1;
```


4ème Partie :

Autres Objets

Objectifs de la partie :

Présenter les vues, Les synonymes, les indexes, et les séquences

Qu'est-ce qu'une VUE ?.

Une vue est une table **virtuelle**, dans laquelle il est possible de rassembler des informations provenant de plusieurs tables.

On parle de "vue" car il s'agit simplement d'une **représentation des données** dans le but d'une **exploitation visuelle**.

Les données présentes dans une vue sont définies grâce à une clause **SELECT**.

Remarque:

Les vues existent en tant qu'objets dans la base de données. Les changements apportés à la structure de la table sous-jacente ne sont pas répercutés dans la vue. Pour inclure des colonnes supplémentaires dans une vue, vous devez redéfinir la vue.

L' Intérêt des VUES.

La vue est une sorte **d'intermédiaire** entre la base de données et l'utilisateur.

Une vue permet ainsi :

- Une **sélection des données** à afficher,
- Une **restriction d'accès** à la table pour l'utilisateur, donc une sécurité des données
- Une **présentation des données** sous plusieurs formes

Types de VUE.

Vue Simple :

- ☐ utilise une seule table
- ne contient ni fonction ni groupe de données
- permet d'exécuter des opérations du LMD

Vue Complexe :

- utilise plusieurs tables
- contient des fonctions ou des groupes de données
- ne permet pas toujours des opérations du LMD

Création des VUES.

CREATE VIEW Vue

(<colonneA>, <colonneB>, <colonneC>, <colonneD>...)

AS SELECT (<colonne1>, <colonne2>, <colonne3>, <colonne4>)

FROM <Nom_table(s)>

WHERE (<condition(s)>)

WITH READ ONLY | WITH CHECK OPTION

- L'option WITH READ ONLY indique qu'il est impossible de réaliser des mises à jour sur la vue.
- L'option WITH CHECK OPTION empêche que l'utilisateur d'ajouter ou modifier dans une vue des lignes non conformes à la définition de la vue.

Suppression des VUES.

Une vue peut être détruite par la commande :

```
DROP VIEW <nom de la vue>
```

Lister les vues :

```
SELECT * FROM USER_VIEWS
```

Exemples de VUES.

Créez des vues qui permettent d'afficher :

1. La liste des employés du département 50,
2. Les employés qui ont été embauché avant l'an 2000,
3. Le salaire des employés du département 10

```
1: CREATE OR REPLACE VIEW vUE1 (nom, prenom) AS  
SELECT last_name, first_name from employees  
WHERE department_id = 50
```

Qu'est-ce qu'un SYNONYM ?

- C'est un **Alias** sur un objet de la base de données, soit un raccourcis.
- L'objet peut être une Table, une Vue, une Séquence, une Procédure, une Fonction, un Package.
- Le SYNONYM peut être **Public** ou **Privé**.
 - **Public** : il sera accessible à partir de tous les schémas ou utilisateurs.
 - **Privé** : il sera accessible uniquement à partir du schéma dans lequel il a été créé.

Intérêts d'un SYNONYM.

- Masquer le vrai nom des objets et leur localisations.
- Simplifier les noms des objets.
- Éviter le pré-fixage dans les requêtes avec le nom de son propriétaire.

(exemple : HR. Employees)

Exemples:

Synonyme	Vue correspondante
cols	User_tab_columns
dict	Dictionnary
ind	User_indexes
obj	User_objects
seq	User_sequences
syn	User_synonyms
tabs	User_tables

Création d'un SYNONYM.

Tout d'abord vous devez avoir le privilège

- Soit **CREATE SYNONYM** pour créer un SYNONYM Privé dans votre schéma,
- Soit **CREATE ANY SYNONYM** pour créer un SYNONYM Privé dans n'importe quel schéma,
- Soit **CREATE PUBLIC SYNONYM** pour créer un SYNONYM Public.

```
CREATE [PUBLIC | ANY] SYNONYM <synonym> FOR <object>  
DROP SYNONYM <synonym>
```

Qu'est-ce qu'un INDEX ?

Un index est un objet de la base de données qui permet **d'accélérer** la recherche des lignes.

Un index peut être créé automatiquement ou manuellement.

- Un index unique est créé automatiquement lors de la définition d'une contrainte Clé Primaire

→ La vue **USER_INDEXES** contient les noms d'index ET

La vue **USER_IND_COLUMNS** contient les noms d'index , des tables et des colonnes

Manipulation d'un INDEX.

Création d'un index :

CREATE [UNIQUE] INDEX <Nom_index> **ON** table (col1, col2, ...)

Exemple:

```
CREATE INDEX idx_employees_lastname ON employees  
(last_name)
```

Suppression d'un index :

DROP INDEX <Nom_index>

Exemple:

```
DROP INDEX idx_employees_lastname
```

Qu'est-ce qu'une SEQUENCE ?

- Une séquence est un objet de base de données Oracle, au même titre qu'une table, une vue, etc...
- Une séquence permet de **Générer** automatiquement des numéros uniques
- Elle Est partageable entre plusieurs utilisateurs et entre plusieurs tables

→ Elle permet de créer une valeur de clé primaire

Création d'une SEQUENCE.

CREATE SEQUENCE

```
[ INCREMENT BY  n ]  
[ START WITH n ]  
[ {MAXVALUE n | NOMAXVALUE} ]  
[ {MINVALUE n | NOMINVALUE} ]  
[ {CYCLE | NOCYCLE} ] [ {CACHE n | NOCACHE} ]
```

- **INCREMENT BY n** : définit l'intervalle entre les numéros
- **START WITH n** : premier numéro de la séquence
- **{MAXVALUE n | NOMAXVALUE}** : valeur maximale
- **{MINVALUE n | NOMINVALUE}** : valeur minimale
- **{CYCLE | NOCYCLE}** : la séquence peut continuer à générer ou non des valeurs
- **{CACHE n | NOCACHE}** : nombre de valeurs pré-allouées et conservées en mémoire.

Interrogation d'une SEQUENCE.

L'interrogation d'une séquence se fait par l'utilisation des "pseudo-colonnes" CURRVAL et NEXTVAL. On parle de **pseudo-colonne** car cela se manipule un peu comme une colonne de table, mais ce n'est pas une colonne de table.

- La pseudo-colonne **CURRVAL** retourne la valeur courante de la séquence.
- La pseudo-colonne **NEXTVAL** incrémente la séquence et retourne la nouvelle valeur.

□ On peut modifier ou supprimer une séquence :

ALTER SEQUENCE <nom_sequence> ...

DROP SEQUENCE <nom_sequence>